

ScamShield v2: The Complete Practical Guide

How to Build & Add New Scam Simulations in Real, Simple Terms (No Prior Experience Needed)

1. Why the Project is Structured This Way (In Plain English)

Welcome! If you are new to the project, you do not need to rewrite any complex web routing or database code. The ScamShield codebase is organized into separate layers so that people can work together without breaking each other's code. Here is why we separate things into different files:

- [*] **Why we separate Visuals from Content:** If we put all the visual buttons, chat boxes, reading articles, and quiz questions into one giant file, the code would become thousands of lines long, messy, and very easy to break. By keeping the visual interface in one file and the written text/quizzes in a separate data file, someone writing quiz questions will never accidentally delete or break the visual interactive buttons!
- [*] **Why we use an Engine Switchboard:** The website needs a simple way to know which visual screen belongs to which scam topic. An engine mapping file connects a simple ID number (like `25`) to your visual code.
- [*] **Why we use a Learning Map:** The home dashboard is where users see the icons for all available lessons. We must add our ID number to this map so a clickable button appears on the user's screen.

To add a completely new simulation, you will edit exactly 4 files in this order:

Step & File Name	Exact File Path	Why You Do This & What It Does
Step 1: Visual Interface	<code>client/src/components/simulators/YourScamSim.jsx</code>	You create a new React component file here. This defines what the user sees on the screen: fake chat messages, websites, and interactive buttons.
Step 2: Content & Quiz	<code>client/src/data/modules.js</code>	You add a new block of text here containing your game ID, headline title, reading lessons for the 'Learn' tab, and multiple-choice questions for the 'Quiz' tab.
Step 3: Engine Link	<code>client/src/components/simulators/ScamSimulatorEngine.jsx</code>	You add two lines here to tell the website: 'When someone asks for Module ID #25, display the visual interface file created in Step 1.'
Step 4: Dashboard Icon	<code>client/src/components/ProgressMap.jsx</code>	You add your ID number into a category list here so an icon shows up on the home screen map for the user to click.

[IMPORTANT NOTE FOR BEGINNERS]: You do **NOT** need to edit core routing files like `ModulePage.jsx`, `App.jsx`, or any backend database code! The website is built to automatically read whatever you put into those 4 files above. Once you follow these 4 steps, your simulation will automatically receive sound effects, XP rewards, and completion badges!

2. Step 1: Creating Your Visual Interface (`simulators/.jsx`)

Why do we do this? Every scam is different. A shopping scam needs a fake store checkout screen, while an investment scam needs a fake chat box. By creating a new React component file inside `client/src/components/simulators/`, you get a blank canvas to build your interactive activity. For example, let's create a file called **CryptoAirdropScamSim.jsx**.

Two Rules You Must Follow in Your Code:

- 1. Why you MUST write 'export default function':** In JavaScript, creating a function inside a file keeps it private by default. If you do not write the keywords `export default` before your function name, the rest of the application is not allowed to import or see your file. The website will crash with a blank screen saying: *'does not contain a default export'*. Writing `export default function` gives the engine permission to load and display your component.
- 2. Why you MUST call 'onComplete(true)' when the user succeeds:** Notice that your function receives a prop called `({ onComplete })`. When the user successfully identifies the scam or completes your activity, your code must execute the function `onComplete(true)`. Why? Because the main web page is waiting for a signal from your code. When you call `onComplete(true)`, the website immediately stops the timer, plays a success sound, marks the step complete, and unlocks the 'Learn' and 'Quiz' tabs at the top of the screen!

Copy-Pasteable Starter Code (CryptoAirdropScamSim.jsx):

```
import React, { useState } from 'react';
import './simulators.css'; // Imports existing standard styling and colors

// RULE 1: 'export default function' is mandatory so the website can read this file!
export default function CryptoAirdropScamSim({ onComplete }) {
  // We use standard React state to track what screen or step the user is on
  const [step, setStep] = useState(0);
  const [feedbackMessage, setFeedbackMessage] = useState("");

  // When the user clicks the button to inspect the smart contract:
  const handleSpotScam = () => {
    setFeedbackMessage("[WARNING]: Malicious smart contract detected! Good job spotting the trap.");
    setStep(1); // Move to the success screen
  };

  // When the user clicks the final button to complete the simulation:
  const handleFinishGame = () => {
    // RULE 2: We call onComplete(true) to tell the website the user succeeded!
    if (onComplete) {
      onComplete(true);
    }
  };

  return (
    <div>
      Scam Simulation: Fake Crypto Giveaway

      {step === 0 ? (
        // STEP 0: The interactive gameplay screen

        <div>
          You receive a Telegram DM: 'Congratulations! Claim 5,000 free USDT on our portal!'

          airdrop-portal-official.net
          The site asks you to 'Connect Wallet' and give unlimited spending approval.
          <button
            onClick={handleSpotScam}
            style={{ background: '#e74c3c', color: '#fff', padding: '10px 15px', border: 'none', cursor: 'pointer' }}
          >
            Inspect Smart Contract Permissions (Spot Scam)
          </button>
        </div>
      ) : (
        // STEP 1: The success screen shown after they spot the scam

        <div>
          {feedbackMessage}
          <button
            onClick={handleFinishGame}
            style={{ background: '#2ecc71', color: '#000', padding: '10px 20px', fontWeight: 'bold', border: 'none', cursor: 'pointer' }}
          >
            Report Scam & Complete Simulation ->
          </button>
        </div>
      )}
    </div>
  );
}
```

3. Step 2: Adding Lesson Content & Quiz (`data/modules.js`)

Why do we do this? After a user plays through your visual screen from Step 1, they need to read theory lessons to understand how the scam works in real life, and take a multiple-choice quiz to test their knowledge. All of this information is stored in one central database file: **client/src/data/modules.js**. Open this file, scroll to the bottom of the list named `export const MODULES = [`, and add a new JavaScript object.

Why you must pick a unique ID number: Every module needs an identifier number so the computer can tell them apart. Check the ID number of the last item in the file and add 1. If the last item was ID 24, your new ID is 25.

Simple Breakdown of Every Required Property:

Property Name	Why We Need It & What You Put Inside
id	Your unique number (e.g., 25). You will use this exact same number in Steps 3 and 4.
title	The headline title displayed at the top of the screen (e.g., 'Fake Crypto Airdrop').
category	The topic category (e.g., 'Advanced Threats', 'Financial Frauds', or 'Shopping Scams').
icon	A text symbol representing your topic (e.g., '\$', 'shield', 'lock', or 'coin').
diff	The difficulty badge. Must be spelled exactly as: 'low', 'med', or 'high'.
desc	A short 1-sentence description shown on the dashboard menu card.
scenario	The background story shown in the header bar above your visual interface.
challenge	The goal instruction telling the user what to do (e.g., 'Inspect the website and find the trap').
exposed	Why we need this: It shows what personal data the user would have leaked in real life if they fell for this scam (e.g., 'Wallet Address', 'Bank Account'). This makes the training feel practical and real!
learn	A list of reading articles (with title and content) displayed in the 'Learn' tab after the user completes the visual activity.
quiz	A list of multiple-choice questions displayed in the 'Quiz' tab. Users answer these to earn XP rewards!

[CRITICAL RULE -- WHY QUIZ ANSWERS START AT 0, NOT 1]: In computer programming, lists always start counting at zero! In your quiz questions below, when you specify which option is the correct answer using `ans: ...`, you must follow this counting system:

- If the correct answer is the **1st** option -> write **ans: 0**
- If the correct answer is the **2nd** option -> write **ans: 1**
- If the correct answer is the **3rd** option -> write **ans: 2**
- If the correct answer is the **4th** option -> write **ans: 3**

Why explain this? Because beginners frequently write `ans: 1` hoping to select the first option, and get frustrated when the website marks their correct answer as wrong!

Complete Metadata Template (Copy and append to the bottom of modules.js):

```

{
  id: 25, // Your unique integer number
  title: "Fake Crypto Airdrop Scam",
  category: "Advanced Threats",
  icon: "$",
  diff: "high",
  desc: "Learn how scammers drain wallets via malicious smart contract approvals.",
  scenario: "You receive a Telegram message claiming you won 5,000 USDT in a community giveaway...",
  challenge: "Inspect the claim portal and identify the dangerous wallet permission request.",

  // 1. DATA EXPOSED LIST (Shows users what data they risked losing)
  exposed: [
    { label: "Wallet Address", val: "0x71C...89A2", risk: "high", desc: "Allows tracking your balance" },
    { label: "Token Approval", val: "Unlimited Spend", risk: "critical", desc: "Gives full control of funds" }
  ],

  // 2. EDUCATIONAL ARTICLES (Shown in the 'Learn' tab after simulation)
  learn: [
    {
      title: "How Wallet Drainer Contracts Work",
      content: "Scammers create replica sites asking you to 'Connect Wallet' and sign a transaction. Instead of sending tokens, the signature grants them allowance to withdraw all your tokens."
    },
    {
      title: "Defensive Checklist",
      content: "Never click links from unsolicited DMs. Always check permissions before signing, and use a separate 'burner' wallet with minimal funds for untested web3 apps."
    }
  ],

  // 3. MULTIPLE CHOICE QUIZ (ans: 1 means the 2nd option in the list is correct!)
  quiz: [
    {
      q: "What does an 'Unlimited Approval' signature allow a smart contract to do?",
      options: [
        "Deposit free reward tokens into your wallet",
        "Withdraw all tokens of that specific cryptocurrency without further permission",
        "Upgrade your wallet security level",
        "Verify your identity on the public ledger"
      ],
      ans: 1, // Look! Index 1 points to the 2nd option above!
      exp: "An unlimited approval grants the external contract ongoing permission to transfer your tokens at any time until you explicitly revoke it on-chain."
    }
  ]
},

```

4. Step 3: Linking the UI to the Engine (`ScamSimulatorEngine.jsx`)

Why do we do this? In Step 1, you created a visual interface file. In Step 2, you created an information object with ID number `25`. However, the website does not automatically know that ID `25` belongs to your new file! Open the file **client/src/components/simulators/ScamSimulatorEngine.jsx**. This file acts as the master bridge. When a user opens Module ID #25, this engine checks its list to find out which file to display.

You only need to add two lines in this file:

Line 1: Tell the file where your new component is stored (Import Statement): At the top of the file with the other imports, add:

```
import ShadowITScamSim from './ShadowITScamSim';
// ADD YOUR NEW COMPONENT IMPORT BELOW:
import CryptoAirdropScamSim from './CryptoAirdropScamSim';
```

Line 2: Connect your ID number to your component in 'SIM_MAP': Scroll down to the dictionary named `const SIM_MAP = {` and add your mapping:

```
const SIM_MAP = {
  0: InstantLoanScamSim,
  1: JobOfferScamSim,
  // ... existing mappings 2 to 23 ...
  24: ShadowITScamSim,
  25: CryptoAirdropScamSim, // <-- BRIDGE YOUR ID 25 TO YOUR COMPONENT HERE!
};
```

5. Step 4: Placing the Icon on the Dashboard Map (`ProgressMap.jsx`)

Why do we do this? Even if your code works perfectly, a user visiting the dashboard (`/dashboard`) needs a clickable button or icon on their screen to select your lesson. Open **client/src/components/ProgressMap.jsx**. This file controls the Duolingo-style learning path map. Notice how lessons are grouped into themed unit categories in a list called `const UNITS = [`.

You have two choices for placing your icon on the screen:

Option A: Put your ID number into an existing Unit category: If your lesson fits into an existing topic, simply add your ID number inside the brackets of `moduleIds`:

```
{
  id: 'advanced',
  title: 'Advanced Threats',
  icon: 'robot',
  description: 'Master emerging threats: deepfakes, crypto scams, and AI attacks',
  color: '#3b82f6',
  moduleIds: [1, 5, 7, 25], // <-- Added ID 25 inside the brackets!
},
```

Option B: Create a brand new Unit category box for your custom lessons: If you want a brand new colored box on the dashboard map, copy and paste a new block into the list:

```
{
  id: 'web3_scams',
  title: 'Web3 & Decentralized Frauds',
```

```

    icon: '$',
    description: 'Learn to protect crypto assets, identify rogue DEXs, and spot NFT traps',
    color: '#00d2d3', // Pick any hex color code for your box!
    moduleIds: [25], // Put your new ID number here
  },

```

6. Bonus: How to Unlock All Games for Testing (Bypass XP)

By default, ScamShield forces users to earn XP and complete games in a specific order before playing new ones. If you are developing a new game, you want to test it instantly without having to play 20 previous levels! You can disable these locks easily.

Open **client/src/components/ProgressMap.jsx** and make these two edits:

Edit 1: Unlock All Category Units: Find the line (around line 94) that calculates `isUnlocked`. It normally looks like `const isUnlocked = unitIndex <= level;`. Change it so it always equals `true`:

```

// Change the lock logic to this:
const isUnlocked = true;

```

Edit 2: Unlock All Individual Games: Scroll down slightly (around line 147) and find the line that calculates `isModLocked`. It normally looks like `const isModLocked = !isActive && !isCompleted;`. Change it so it always equals `false`:

```

// Change the module lock logic to this:
const isModLocked = false;

```

Save the file. Your website will now ignore XP levels and completion history, making every single game instantly clickable!

7. Common Pitfalls to Avoid (Read Before Testing!)

Here are the top 5 practical mistakes that trip up new developers, explained simply so you can fix them:

Mistake / Pitfall	Why It Happens & How to Fix It Easily
1. Forgetting 'export default function' (Most Common!)	Why it happens: In Step 1, if you write function <code>MySim()</code> instead of export default function <code>MySim()</code> , JavaScript keeps your function private. When the engine tries to import it, the website crashes with a blank screen and says: <i>'does not contain a default export'</i> . Always make sure those two words are written at the top of your UI file!
2. Quiz Answers Counting from 1 Instead of 0	Why it happens: In Step 2, if your quiz question has 4 options, remember that option #1 is index 0, option #2 is index 1, option #3 is index 2, and option #4 is index 3. If you write <code>ans: 1</code> hoping to select the first option, the website will mark option #1 as wrong!

3. Mismatched ID Numbers Across Files	Why it happens: If you write id: 25 in modules.js, but write 26: MySim in ScamSimulatorEngine.jsx, the bridge connects to the wrong number and the screen will not load. Ensure the exact same number is used in Steps 2, 3, and 4!
4. Missing Commas Between Items	Why it happens: In JavaScript data files like modules.js, every object block in a list must be separated by a comma (.). When you paste your new lesson at the bottom of the array, make sure there is a comma separating the previous lesson block from your new one!
5. Case Sensitivity in Filenames	Why it happens: If you save your file as myScamSim.jsx (with a small 'm'), but write import MyScamSim from './MyScamSim' (with a capital 'M'), the website might work on Windows but will crash on production web servers! Always match uppercase and lowercase letters exactly.

8. Step-by-Step Testing Guide

Once you have edited your files, follow this simple checklist to verify your simulation works:

- [*] **[TEST 1] Start the Server:** Open your terminal inside the client/ folder and type the command `npm run dev`. Check that no red syntax error text appears in the terminal.
- [*] **[TEST 2] Check the Dashboard (`/dashboard`):** Open your web browser to `http://localhost:5173/dashboard`. Scroll to your Unit category box and verify your new game icon is displayed.
- [*] **[TEST 3] Click and Launch:** Click your new game icon. Verify that the header bar displays your title and scenario text from Step 2.
- [*] **[TEST 4] Complete the Activity:** Click through your visual screen. When you finish, check that calling `onComplete(true)` automatically unlocks the **Learn** and **Quiz** tabs at the top of the screen.
- [*] **[TEST 5] Take the Quiz:** Click into the Quiz tab, answer your multiple-choice questions, and verify that the green '+XP' popup notification appears when you finish!